

ASICI

Asociación Internacional
de Ciberseguridad



Manual básico para interpretar access.log de un servidor web

1. ¿Qué es el **access.log**?

El archivo **access.log** es un registro generado por servidores web como **Apache HTTP Server**, **Nginx**, **LiteSpeed** u otros. Su función principal es guardar cada solicitud HTTP que recibe el servidor.

En este log podemos ver:

- Quién se conectó
- Cuándo se conectó
- Qué recurso solicitó
- Qué método HTTP usó
- Qué código de respuesta recibió
- Cuántos bytes se enviaron
- Desde dónde llegó la visita
- Qué navegador o herramienta usó

Este tipo de log es muy importante para:

- Monitoreo de tráfico web
- Análisis de errores
- Investigación de incidentes
- Detección de ataques
- Análisis forense
- Auditoría de accesos
- Entrenamiento de modelos de Machine Learning

2. Tipo de log: Combined Log Format

Uno de los formatos más usados en Apache y Nginx es el **Combined Log Format**.

Ejemplo:

```
192.168.1.50 - - [02/May/2026:10:15:23 -0500] "GET /login.php HTTP/1.1" 200 3456  
"https://empresa.com/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"
```

Este formato combina información básica de acceso con datos adicionales como:

- Referer
- User-Agent

3. Estructura general de una línea **access.log**

Una línea típica tiene esta estructura:

IP - usuario [fecha:hora zona] "METODO URL PROTOCOLO" CODIGO BYTES
"REFERER" "USER_AGENT"

Ejemplo descompuesto:

Campo	Ejemplo	Significado
IP origen	192.168.1.50	Dirección IP del cliente que hizo la solicitud
Identidad remota	-	Campo histórico, normalmente vacío
Usuario autenticado	-	Usuario si hubo autenticación HTTP
Fecha y hora	[02/May/2026:10:15:23 -0500]	Momento exacto de la solicitud
Método HTTP	GET	Acción solicitada al servidor
URL solicitada	/login.php	Recurso pedido
Protocolo	HTTP/1.1	Versión del protocolo HTTP
Código HTTP	200	Resultado de la solicitud
Bytes	3456	Tamaño de la respuesta
Referer	https://empresa.com/	Página desde donde vino el usuario

User-Agent	Mozilla/5.0...	Navegador, bot o herramienta usada
------------	----------------	------------------------------------

4. Explicación de cada campo

4.1 IP de origen

Ejemplo:

185.220.101.45

Representa la dirección IP desde donde se hizo la solicitud.

Puede ser:

Usuario legítimo

Bot de buscador

Atacante

Proxy

VPN

Tor

Herramienta automática

Indicadores sospechosos:

Muchas solicitudes desde una misma IP

Muchas solicitudes fallidas

Acceso repetido a rutas sensibles

IPs de países no esperados

IPs con User-Agent extraño

4.2 Fecha y hora

Ejemplo:

[02/May/2026:10:15:23 -0500]

Permite analizar:

Picos de tráfico

Ataques por horario

Secuencia de eventos
Correlación con otros logs

Partes:

Elemento	Ejemplo
Día	2
Mes	May
Año	2026
Hora	10:15:23
Zona horaria	-500

4.3 Método HTTP

El método HTTP indica la acción que el cliente quiere realizar.

Método	Significado	Interpretación
GET	Solicita un recurso	Normal en navegación
POST	Envía datos al servidor	Normal en formularios, login, APIs
PUT	Sube o reemplaza recursos	Sospechoso si no está autorizado
DELETE	Elimina recursos	Sospechoso en sitios públicos
HEAD	Solicita solo cabeceras	Puede ser normal o usado para reconocimiento
OPTIONS	Consulta métodos permitidos	Puede indicar reconocimiento

PATCH	Modifica parcialmente un recurso	Normal en APIs, sospechoso en sitios simples
TRACE	Método de diagnóstico	Generalmente debería estar deshabilitado

Ejemplo:

"GET /index.html HTTP/1.1"

Ejemplo sospechoso:

"OPTIONS /admin HTTP/1.1"

5. Códigos HTTP que se deben aprender

Los códigos HTTP son claves para interpretar el comportamiento del servidor.

5.1 Códigos 2xx: respuestas exitosas

Código	Nombre	Significado
200	OK	Solicitud exitosa
201	Created	Recurso creado
204	No Content	Solicitud exitosa sin contenido

Ejemplo:

"GET /index.html HTTP/1.1" 200 4521

Interpretación:

El recurso fue entregado correctamente.

5.2 Códigos 3xx: redirecciones

Código	Nombre	Significado
301	Moved Permanently	Redirección permanente
302	Found	Redirección temporal
304	Not Modified	Recurso en caché, no fue modificado

Ejemplo:

"GET / HTTP/1.1" 301 178

Interpretación:

El cliente fue redirigido a otra URL.

5.3 Códigos 4xx: errores del cliente

Código	Nombre	Significado	Posible lectura de seguridad
400	Bad Request	Solicitud malformada	Puede ser tráfico anómalo
401	Unauthorized	Requiere autenticación	Intento de acceso sin credenciales
403	Forbidden	Acceso prohibido	Intento a recurso restringido
404	Not Found	Recurso no encontrado	Escaneo de rutas o fuerza bruta de directorios
405	Method Not Allowed	Método no permitido	Uso de métodos peligrosos

408	Request Timeout	Tiempo agotado	Cliente lento o ataque tipo slow request
429	Too Many Requests	Demasiadas solicitudes	Posible abuso o fuerza bruta

Ejemplo:

"GET /admin/config.php HTTP/1.1" 404 512

Interpretación:

Puede ser un intento de encontrar archivos sensibles.

5.4 Códigos 5xx: errores del servidor

Código	Nombre	Significado	Posible lectura de seguridad
500	Internal Server Error	Error interno	Puede revelar fallos por payload malicioso
502	Bad Gateway	Error de gateway	Problema con backend/proxy
503	Service Unavailable	Servicio no disponible	Saturación o posible DoS
504	Gateway Timeout	Timeout de gateway	Backend lento o ataque volumétrico

Ejemplo:

"POST /api/login HTTP/1.1" 500 1024

Interpretación:

La solicitud provocó un error interno. Si se repite, puede indicar explotación o fuzzing.

6. Campos importantes para análisis de seguridad

6.1 URL solicitada

Ejemplo:

/wp-admin/

La URL ayuda a identificar qué recurso fue atacado.

Rutas sensibles comunes:

Ruta	Posible significado
/admin	Panel administrativo
/login	Página de autenticación
/wp-admin	WordPress admin
/phpmyadmin	Administración de base de datos
/.env	Archivo de configuración sensible
/config.php	Archivo de configuración
/backup.zip	Copia de respaldo expuesta
/server-status	Estado interno del servidor
/.git/config	Repositorio Git expuesto

6.2 User-Agent

Ejemplo normal:

Mozilla/5.0 (Windows NT 10.0; Win64; x64) Chrome/124.0

Ejemplos sospechosos:

sqlmap
nikto
masscan
nmap
curl
wget
python-requests
Go-http-client

El User-Agent puede revelar herramientas automáticas, bots o scripts.

6.3 Referer

Ejemplo:

"https://empresa.com/inicio"

Puede ayudar a saber desde dónde llegó la solicitud.

Casos sospechosos:

Referer vacío en muchas solicitudes automatizadas
Referer externo extraño
Referer inconsistente con la navegación normal

6.4 Bytes enviados

Ejemplo:

3456

Puede ayudar a identificar:

Respuestas muy grandes
Descarga masiva de archivos
Exfiltración
Errores con tamaño anómalo

7. Tabla de patrones útiles para detectar ataques

Tipo de ataque	Indicadores en access.log
SQL Injection	Parámetros con ', ", UNION SELECT, OR 1=1, sleep, benchmark
XSS	Parámetros con <script>, onerror=, javascript:
Path Traversal	Uso de ../, ..%2f, /etc/passwd, boot.ini
Directory Brute Force	Muchas solicitudes 404 a rutas diferentes
Fuerza bruta login	Muchos POST /login con 401, 403 o 429
Escaneo de WordPress	Accesos a /wp-admin, /wp-login.php, /xmlrpc.php
Búsqueda de archivos sensibles	Acceso a /.env, /config.php, /backup.zip, /.git/config
Reconocimiento	Métodos HEAD, OPTIONS, muchos 404

Bot malicioso	User-Agent como sqlmap, nikto, curl, python-requests
DoS básico	Muchas solicitudes por segundo desde una o varias IPs
Abuso de API	Muchas llamadas a /api/, códigos 429, errores repetidos

8. Ejemplos de líneas normales

Acceso normal a página principal

203.0.113.10 - - [02/May/2026:09:12:10 -0500] "GET /index.html HTTP/1.1" 200 5421
"https://google.com/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"

Interpretación:

Una IP accedió correctamente a la página principal.

Envío de formulario

203.0.113.15 - - [02/May/2026:09:14:33 -0500] "POST /contacto HTTP/1.1" 200 2310
"https://empresa.com/contacto" "Mozilla/5.0 (Macintosh; Intel Mac OS X)"

Interpretación:

Un usuario envió información mediante un formulario.

9. Ejemplos de líneas sospechosas

SQL Injection

185.220.101.45 - - [02/May/2026:10:20:11 -0500] "GET /producto?id=1' OR '1'='1 HTTP/1.1"
500 1200 "-" "sqlmap/1.7"

Señales:

Parámetro sospechoso en la URL

User-Agent sqlmap

Código 500

Path Traversal

198.51.100.23 - - [02/May/2026:10:25:44 -0500] "GET /download?file=../../../../etc/passwd HTTP/1.1" 403 740 "-" "curl/8.0"

Señales:

Uso de ../

Referencia a /etc/passwd

Código 403

User-Agent automatizado

Escaneo de rutas sensibles

203.0.113.90 - - [02/May/2026:10:30:01 -0500] "GET /.env HTTP/1.1" 404 512 "-" "python-requests/2.31"

Señales:

Intento de acceder a archivo sensible

Código 404

User-Agent automatizado

Fuerza bruta contra login

192.0.2.77 - - [02/May/2026:10:35:07 -0500] "POST /login HTTP/1.1" 401 890 "https://empresa.com/login" "Mozilla/5.0"

Una sola línea no confirma fuerza bruta. Se confirma mejor con muchas líneas similares:

Misma IP

Mismo endpoint /login

Muchos POST

Muchos códigos 401, 403 o 429

Poco tiempo entre intentos

10. Cómo identificar ataques en **access.log**

10.1 Buscar muchas respuestas 404

Muchos errores 404 desde la misma IP pueden indicar escaneo.

Ejemplo de lógica:

IP 185.10.10.20 genera 300 errores 404 en 5 minutos

Interpretación:

Probable directory brute force o escaneo de rutas.

10.2 Buscar errores 401 y 403 en login

Indicadores:

POST /login

POST /wp-login.php

POST /admin/login

Códigos 401, 403 o 429

Repetición desde la misma IP

Interpretación:

Posible fuerza bruta o password spraying.

10.3 Buscar patrones maliciosos en la URL

Patrones comunes:

' OR '1'='1

UNION SELECT

<script>

../

/etc/passwd

cmd=

exec=

base64

%2e%2e%2f

Interpretación:

Intentos de inyección, XSS, path traversal o ejecución remota.

10.4 Revisar User-Agent sospechoso

User-Agents que pueden indicar automatización:

sqlmap
nikto
nmap
masscan
curl
wget
python-requests
Go-http-client
Java

Importante:

curl o python-requests no siempre son maliciosos.
También pueden ser usados por integraciones legítimas.
Deben analizarse con IP, URL, frecuencia y código HTTP.

10.5 Buscar métodos HTTP inusuales

Métodos que merecen atención:

PUT
DELETE
TRACE
OPTIONS
PATCH

Ejemplo:

"PUT /uploads/shell.php HTTP/1.1" 403 900

Interpretación:

Intento de subir o modificar un archivo en el servidor.

10.6 Identificar picos de tráfico

Indicadores:

Muchas solicitudes por segundo
 Muchas IPs contra un mismo endpoint
 Aumento de códigos 503
 Aumento de códigos 429

Interpretación:

Posible DoS, scraping agresivo o abuso automatizado.

11. Tabla resumen para analistas

| Señal | Posible ataque |
|-------------------------------|-----------------------------|
| Muchos 404 | Escaneo de directorios |
| Muchos 401 en /login | Fuerza bruta |
| Muchos 403 en rutas sensibles | Intentos no autorizados |
| UNION SELECT en URL | SQL Injection |
| <script> en URL | XSS |
| ../ en URL | Path Traversal |
| Acceso a /.env | Búsqueda de secretos |
| Acceso a /.git/config | Exposición de repositorio |
| User-Agent sqlmap | SQL Injection automatizado |
| User-Agent nikto | Escaneo de vulnerabilidades |

| | |
|-------------------------|---------------------------------------|
| Método OPTIONS repetido | Reconocimiento |
| Método PUT o DELETE | Intento de modificación no autorizada |
| Muchos 500 | Fuzzing o intento de explotación |
| Muchos 429 | Rate limit activado |

12. Recomendaciones para análisis con Pandas

Cuando cargues el log en Pandas, conviene extraer estas columnas:

ip
datetime
method
url
protocol
status_code
bytes
referer
user_agent
hour
date
is_error
is_suspicious
attack_type

Consultas básicas recomendadas:

Top 10 IPs con más solicitudes
Top 10 URLs más solicitadas
Distribución de códigos HTTP
Errores 4xx y 5xx por hora
IPs con más 404
IPs con más 401
User-Agents más frecuentes
Métodos HTTP usados

Solicitudes por minuto
Eventos sospechosos por tipo

13. Conclusión

El `access.log` es una de las fuentes más importantes para detectar actividad sospechosa en servidores web. No solo permite saber quién accedió al sitio, sino también identificar patrones de ataque como fuerza bruta, escaneo, inyección SQL, XSS, path traversal y abuso de APIs.

La clave no está en una sola línea del log, sino en correlacionar:

- IP
- Frecuencia
- URL
- Método HTTP
- Código de respuesta
- User-Agent
- Hora del evento
- Patrones en parámetros

Un buen analista debe aprender primero a leer manualmente el log, luego usar Pandas para agrupar y filtrar, y finalmente aplicar visualización y Machine Learning para detectar patrones más complejos.